

CPU Software Renderer in C

In this challenge, you will build a **very small software renderer** entirely on the CPU, using pure C.

You are **not allowed** to use:

- OpenGL
- Vulkan
- DirectX
- SDL rendering APIs
- Any graphics engine

Instead, you will manually:

- allocate a **framebuffer** in memory
- manipulate pixels directly
- rasterize primitives
- export the result to a BMP image

The final output of your program should be a valid:

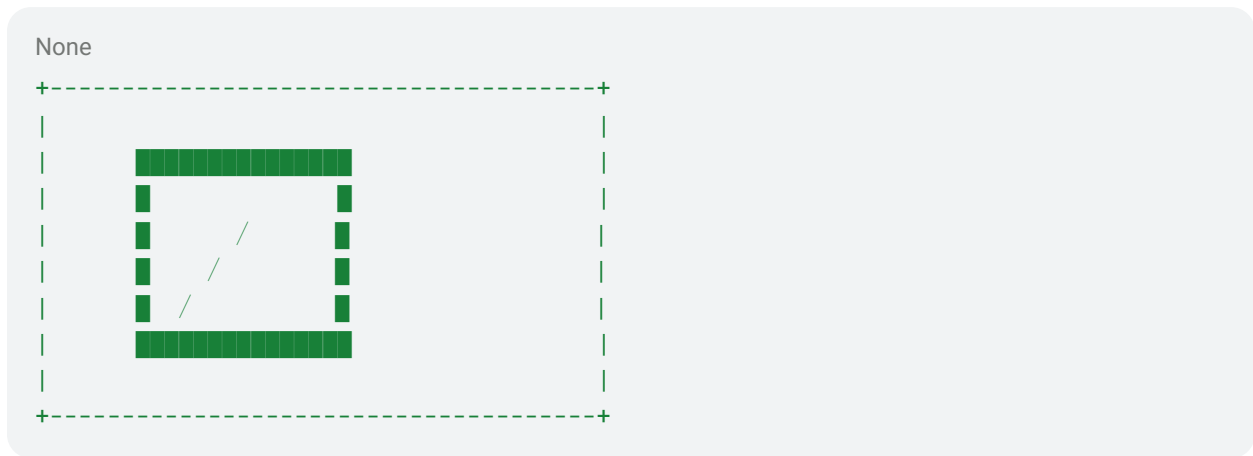
None

`output.bmp`

containing rendered shapes.

Goal

Generate an image similar to:



Using only CPU-side memory manipulation.

What You Must Implement

1. Framebuffer

Create a pixel buffer in memory:

None

```
uint32_t *pixels;
```

Each pixel is a 32-bit color.

Example:

None

```
800 x 600
```

2. Pixel Plotting

Implement:

```
None  
void draw_pixel(int x, int y, uint32_t color);
```

This is the foundation of everything else.

3. Rectangle Rendering

Implement:

```
None  
void draw_rect(int x, int y, int w, int h, uint32_t color);
```

You may draw:

- outlined rectangles or
 - filled rectangles
-

4. Line Rendering

Implement line rasterization manually.

Suggested:

- [Bresenham algorithm](#)

Function:

```
None  
void draw_line(int x0, int y0, int x1, int y1, uint32_t color);
```

5. BMP Export

Write the framebuffer into a valid BMP image.

The program should produce:

```
None
output.bmp
```

that can be opened by normal image viewers.

Allowed

- Standard C library
- `stdint.h`
- `stdio.h`
- `stdlib.h`
- `string.h`

Suggested Milestones

Milestone 1

Generate:

- black image

Milestone 2

Plot random pixels.

Milestone 3

Draw rectangles.

Milestone 4

Draw lines.

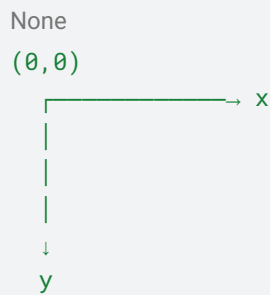
Milestone 5

Export valid BMP.

Important Technical Details

Coordinate System

Use:



Top-left origin.

Color Format

Use:

None

0x00RRGGBB

Example:

None

0x00FF0000 // red
0x0000FF00 // green
0x000000FF // blue

Expected Learning Outcomes

By the end, you should understand:

- how images are stored in memory
- how pixels are addressed
- how drawing primitives work internally
- how binary image formats work
- why memory layout matters
- how renderers operate fundamentally

Starter Code

None

```
#include <stdint.h>
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define WIDTH 800
#define HEIGHT 600

typedef struct
{
    int width;
    int height;
    uint32_t *pixels;
} Framebuffer;

static Framebuffer framebuffer;

void clear_screen(uint32_t color)
{
    for (int i = 0; i < framebuffer.width * framebuffer.height; i++)
    {
        framebuffer.pixels[i] = color;
    }
}

void draw_pixel(int x, int y, uint32_t color)
{
    if (x < 0 || x >= framebuffer.width ||
        y < 0 || y >= framebuffer.height)
    {
        return;
    }

    framebuffer.pixels[y * framebuffer.width + x] = color;
}

void draw_rect(int x, int y, int w, int h, uint32_t color)
{
    for (int py = y; py < y + h; py++)
    {
        for (int px = x; px < x + w; px++)
        {
            draw_pixel(px, py, color);
        }
    }
}
```

```

/*
    TODO:
        Implement Bresenham line drawing
*/
void draw_line(int x0, int y0, int x1, int y1, uint32_t color)
{
}

/*
    VERY SIMPLE BMP WRITER

    Writes framebuffer into a 32-bit BMP file.
*/
void write_bmp(const char *filename)
{
    FILE *f = fopen(filename, "wb");

    if (!f)
    {
        perror("fopen");
        return;
    }

    int filesize = 54 + framebuffer.width * framebuffer.height * 4;

    unsigned char file_header[14] = {
        'B', 'M',
        0, 0, 0, 0,
        0, 0,
        0, 0,
        54, 0, 0, 0
    };

    unsigned char info_header[40] = {
        40, 0, 0, 0,
        0, 0, 0, 0,
        0, 0, 0, 0,
        1, 0,
        32, 0
    };

    file_header[2] = (unsigned char)(filesize);
    file_header[3] = (unsigned char)(filesize >> 8);
    file_header[4] = (unsigned char)(filesize >> 16);

```

```

file_header[5] = (unsigned char)(filesize >> 24);

info_header[4] = (unsigned char)(framebuffer.width);
info_header[5] = (unsigned char)(framebuffer.width >> 8);
info_header[6] = (unsigned char)(framebuffer.width >> 16);
info_header[7] = (unsigned char)(framebuffer.width >> 24);

info_header[8] = (unsigned char)(framebuffer.height);
info_header[9] = (unsigned char)(framebuffer.height >> 8);
info_header[10] = (unsigned char)(framebuffer.height >> 16);
info_header[11] = (unsigned char)(framebuffer.height >> 24);

fwrite(file_header, 1, 14, f);
fwrite(info_header, 1, 40, f);

fwrite(
    framebuffer.pixels,
    4,
    framebuffer.width * framebuffer.height,
    f
);

fclose(f);
}

int main(void)
{
    framebuffer.width = WIDTH;
    framebuffer.height = HEIGHT;

    framebuffer.pixels = malloc(
        WIDTH * HEIGHT * sizeof(uint32_t)
    );

    if (!framebuffer.pixels)
    {
        fprintf(stderr, "Failed to allocate framebuffer\n");
        return 1;
    }

    clear_screen(0x00202020);

    draw_rect(100, 100, 300, 200, 0x0000FF00);

    draw_line(100, 100, 400, 300, 0x00FFFFFF);

```

```
write_bmp("output.bmp");  
  
free(framebuffer.pixels);  
  
return 0;  
}
```

Build Command

```
None  
gcc -Wall -Wextra -pedantic -std=c99 main.c -o renderer
```

Run:

```
None  
./renderer
```

Then open:

```
None  
output.bmp
```